

Complex Computing Problem (CCP)
Course Code & Title: CSC-203 Computer Networks

1. Course Learning Outcome

CLO 3: Analyze network protocols based on network performance requirements at the application and transport layers.

Level of CLO: CLO3 is mapped to the 4th level, i.e., Analyze of Bloom's taxonomy.

2. CCP Attributes

- A2 Depth of Analysis Required: Has no obvious solution and requires conceptual thinking and innovative analysis to formulate suitable abstract models.
- A3 Depth of Knowledge Required: A solution requires the use of in-depth computing or domain knowledge and an analytical approach that is based on well-founded principles.
- A8 Interdependence: Is a high-level problem possibly including many component parts or sub-problems.

3. Problem Statement

Design and develop a real-time Collaborative Network Monitoring and Alert Dashboard using socket programming that enables multiple connected clients (more than three devices) to monitor, share, and respond to simulated network events over a local area network (LAN/Wi-Fi).

Problem Description

The application must support the following core features:

1. Multi-client connectivity supporting three or more devices communicating over the same local network (LAN/Wi-Fi).
2. Real-time broadcasting of network status updates and alert notifications to all connected clients.
3. Display of all currently active client nodes connected to the central monitoring server, including their host names and IP addresses.
4. Event log sharing, clients can upload and distribute text-based log files, screenshots, and short video recordings of simulated faults between all connected nodes.
5. Group monitoring rooms allowing clients to subscribe to specific alert categories (e.g., CPU alerts, bandwidth alerts, security alerts), with the ability to join or leave a room at any time.
6. Private direct communication channel between any two selected clients for isolated incident discussion and coordination.
7. A simulated voice/notification alert feature that broadcasts an audible or text-based emergency alert to all connected clients when a critical network threshold is breached.

The system must be developed using Python socket programming and should demonstrate efficient use of threads for handling multiple concurrent clients. Students are encouraged to design the application in a modular and scalable manner, similar to real-world network operations center (NOC) tools such as Nagios or PRTG Network Monitor.

4. Instructions

- You may work in a group of three members.
- Your project will be graded according to the provided rubrics.
- Deadline to submit: June 07, 2026

5. Deliverables

Students are required to submit the following:

1. Source Code

- Complete and well-documented Python code (client-server architecture using sockets and threading).

2. Project Report

- Introduction and problem overview
- System architecture (client-server model and node topology)
- Protocol design (application & transport layer considerations, choice of TCP/UDP)
- Implementation details (threads, sockets, alert handling, event broadcasting)
- Challenges faced and solutions applied
- Testing and performance analysis

3. User Interface (UI)

- Functional interface (CLI or GUI) supporting dashboard operations: group rooms, private channels, and event log display.

4. Demonstration / Presentation

- Live demo showing multiple clients connected to the monitoring server.
- Demonstration of group alert rooms, private messaging, and file/log sharing.

5. Testing Evidence

- Screenshots or logs showing multiple clients connected and interacting simultaneously.
- Performance observations (latency, concurrency handling, packet loss simulation, etc.).

6. Alignment with Attributes of Complex Computing Problems

Attribute	Alignment / Justification
Depth of Analysis Required	Students must analyze the communication and synchronization requirements of a distributed monitoring system and design appropriate event-driven protocols. No direct solution exists; requires decisions on alert propagation logic, threshold handling, and concurrent state management.
Depth of Knowledge Required	Requires strong understanding of socket programming, transport layer protocols (TCP/UDP), multi-threading, and network event handling. Students must apply theoretical networking concepts to a practical, time-sensitive implementation.

Interdependence	The system consists of multiple interdependent components: monitoring server, multiple client nodes, alert broadcasting engine, room subscription manager, threading model, and communication protocols. Proper coordination among all components is essential to ensure correctness, reliability, and real-time responsiveness.
------------------------	--

7. Complex Computing Problem Assessment Rubrics

Criteria (Marks)	Description	Excellent	Good	Satisfactory	Needs Improvement
Functionality Implementation (2 Marks)	Implementation of all required features: multi-client monitoring, real-time alert broadcasting, emergency notifications, and active node display.	All features fully functional without errors.	Most features work correctly with minor issues.	Basic communication works but with major limitations.	Application fails to run or lacks core features.
Group Rooms & Private Channel (2 Marks)	Implementation of both group alert rooms (subscription-based) and one-to-one private communication channels.	Both modes implemented perfectly and work seamlessly.	Implemented but minor bugs exist.	Only one mode works correctly.	Not implemented or non-functional.
File & Log Sharing (1 Mark)	Ability to send text logs, screenshots, and short video recordings between clients.	All supported file types transfer successfully.	Some file types transfer properly.	File transfer is partially functional.	Feature not implemented.
Thread Management & Network Efficiency (2 Marks)	Use of threads to handle multiple client connections efficiently without crashing or significant lag.	Excellent concurrency handling, smooth multi-client performance.	Handles most connections well.	Occasional lag or client disconnects.	Poor threading, frequent crashes.
Code Structure & Documentation (2 Marks)	Code readability, modular design, and inline comments explaining logic and design decisions.	Clean, modular, and well-documented code.	Mostly organized with some comments.	Understandable but lacks structure.	Poorly structured or undocumented code.
Optional Feature Integration (1 Mark)	Implementation of advanced features such as encryption (TLS/SSL), a graphical UI, or performance metrics dashboard.	One or more advanced features implemented effectively.	Attempted but incomplete.	Feature mentioned but not functional.	No attempt made.